

Step-by-Step Project Walkthrough

OpenAlex DBMS Comparison: PostgreSQL 16 vs Neo4j 5

Students Hamza Abdul Kader, 2230150
Leonardo Marasca, 2217140
Purpose Simple explanation for presenting and discussing the project
Dataset OpenAlex AI/RAG research paper subset
Systems PostgreSQL 16 and Neo4j 5

This document explains what we did in chronological order, in simple words.

Contents

1 One-Sentence Summary	3
2 What the Project Is About	3
3 What We Chose and Why	3
4 Repository Structure	3
5 Step 1: Collecting Data from OpenAlex	4
6 Step 2: Cleaning and Normalizing the Data	5
6.1 Important Cleaning Problems	5
7 Step 3: Dataset Size	5
8 Step 4: Running the Databases with Docker	6
9 Step 5: PostgreSQL Implementation	6
9.1 Loading PostgreSQL	7
10 Step 6: Neo4j Implementation	7
10.1 Loading Neo4j	8
11 Step 7: Validating the Load	8
12 Step 8: Writing Equivalent Queries	8
12.1 Example Query	9
13 Step 9: Benchmarking	9
14 Step 10: Benchmark Results	10
15 Step 11: Visual Evidence	10
16 Step 12: Final Deliverables	11
17 How to Explain the Project in the Presentation	11
18 What Leo Should Know Before Presenting	11
19 How This Connects to the Lecture Slides	12

20 Possible Professor Questions and Short Answers	13
21 Commands to Reproduce the Project	13
22 Final Simple Conclusion	13

1 One-Sentence Summary

We built a project that collects AI-related research paper data from OpenAlex, loads the same cleaned dataset into PostgreSQL and Neo4j, runs equivalent SQL and Cypher queries, benchmarks them, and compares which database model is better for each type of question.

2 What the Project Is About

The project is a DBMS comparison. We compare:

- **PostgreSQL 16**: a relational database, based on tables and SQL;
- **Neo4j 5**: a graph database, based on nodes, relationships, and Cypher.

The dataset is about AI research papers from OpenAlex. This is a good dataset because research papers naturally have many relationships:

- authors write papers;
- papers have topics;
- papers cite other papers;
- authors can be connected through citations, shared papers, or shared topics.

This makes it suitable for comparing a relational model and a graph model.

3 What We Chose and Why

At the beginning, we had to choose a project topic and a dataset. We selected OpenAlex because:

- it is open and public;
- it has an API;
- it contains scholarly works, authors, topics, and citations;
- it is related to AI and RAG, which makes the topic more interesting.

The official links used in the proposal were:

- OpenAlex website: <https://openalex.org/>
- OpenAlex data documentation: <https://help.openalex.org/hc/en-us/articles/24397285563671-About->

The professor approved the proposal, so we continued with this topic.

4 Repository Structure

The repository is organized like this:

```
src/  
  collect_openalex.py  
  benchmark_queries.py  
  generate_benchmark_charts.py  
  
database/postgres/  
  schema.sql  
  load.sql  
  queries.sql  
  
database/neo4j/  
  constraints.cypher  
  import.cypher  
  queries.cypher  
  
benchmarks/  
  queries.json  
  results/benchmark_results.csv  
  
docs/  
  project_report.pdf  
  OpenAlex_DBMS_Comparison_Sapienza.pptx  
  figures/
```

The important idea is:

- `src/` contains Python scripts;
- `database/` contains database schema, load, and query files;
- `benchmarks/` contains query definitions and results;
- `docs/` contains the report, presentation, and screenshots.

5 Step 1: Collecting Data from OpenAlex

The data collection script is:

```
src/collect_openalex.py
```

We run it with:

```
python src/collect_openalex.py --per-term 75 --per-page 75
```

The script searches OpenAlex for AI-related terms:

- retrieval augmented generation;
- large language models;

- artificial intelligence;
- machine learning.

It saves the raw API result as:

```
data/raw/openalex_works.jsonl
```

JSONL means one JSON object per line. This is useful because OpenAlex returns nested JSON data.

6 Step 2: Cleaning and Normalizing the Data

OpenAlex data is nested. For example, a paper object can contain authors, topics, citations, abstract information, and metadata all inside one JSON object.

Databases are easier to load if the data is normalized into separate CSV files. So the script creates:

```
data/processed/papers.csv
data/processed/authors.csv
data/processed/topics.csv
data/processed/paper_authors.csv
data/processed/paper_topics.csv
data/processed/citations.csv
```

6.1 Important Cleaning Problems

We solved these problems:

- **Nested JSON:** converted into flat CSV files.
- **Abstract inverted index:** reconstructed abstracts into readable text.
- **Duplicate relationships:** removed duplicate paper-author pairs.
- **External citations:** kept only citations where both papers exist in our selected subset.

This step is important because it is the ETL part of the project:

```
Extract: OpenAlex API
Transform: clean and normalize JSON
Load: import CSV files into PostgreSQL and Neo4j
```

7 Step 3: Dataset Size

The current dataset contains:

Data element	Count
Papers	299
Authors	1966
Topics	1104
Paper-author relationships	2077
Paper-topic relationships	4288
Citation relationships	392

Table 1: Current dataset size

This is not the full OpenAlex database. It is a controlled subset, small enough to run locally but rich enough to compare relationships.

8 Step 4: Running the Databases with Docker

We use Docker Compose so the project can run locally in a reproducible way.

The services are:

- PostgreSQL 16;
- Neo4j 5;
- Adminer, a web interface for PostgreSQL.

Start everything with:

```
docker compose up -d
```

Check that everything is running:

```
docker compose ps
```

Stop everything with:

```
docker compose down
```

9 Step 5: PostgreSQL Implementation

PostgreSQL uses a relational model. That means data is stored in tables.

The schema is in:

```
database/postgres/schema.sql
```

The tables are:

- `papers`;

- authors;
- topics;
- paper_authors;
- paper_topics;
- citations.

9.1 Loading PostgreSQL

Run:

```
docker exec openalex-postgres psql -U openalex -d openalex_ai -f /sql/schema.sql
docker exec openalex-postgres psql -U openalex -d openalex_ai -f /sql/load.sql
```

Adminer is used to visually inspect PostgreSQL:

<http://localhost:8080>

Login:

```
System: PostgreSQL
Server: postgres
Username: openalex
Password: openalex
Database: openalex_ai
```

10 Step 6: Neo4j Implementation

Neo4j uses a graph model. That means data is stored as nodes and relationships.

Nodes:

- Paper;
- Author;
- Topic.

Relationships:

- (Author)-[:AUTHORED]->(Paper);
- (Paper)-[:HAS_TOPIC]->(Topic);
- (Paper)-[:CITES]->(Paper).

10.1 Loading Neo4j

Run:

```
docker exec openalex-neo4j cypher-shell -u neo4j -p openalex123 -f /cypher/constraints.cypher
docker exec openalex-neo4j cypher-shell -u neo4j -p openalex123 -f /cypher/import.cypher
```

Neo4j Browser is available at:

<http://localhost:7474>

Login:

Username: neo4j

Password: openalex123

11 Step 7: Validating the Load

After loading both databases, we checked that the counts match.

Data element	PostgreSQL	Neo4j
Papers	299	299
Authors	1966	1966
Topics	1104	1104
Paper-author / AUTHORED	2077	2077
Paper-topic / HAS_TOPIC	4288	4288
Citations / CITES	392	392

Table 2: Validation counts

This is important because if the counts did not match, the benchmark would be unfair. The benchmark must compare the same logical dataset in both systems.

12 Step 8: Writing Equivalent Queries

We created 11 paired queries. Each query has:

- a PostgreSQL SQL version;
- a Neo4j Cypher version.

The paired workload is stored in:

`benchmarks/queries.json`

The manual SQL queries are also in:

database/postgres/queries.sql

The manual Cypher queries are in:

database/neo4j/queries.cypher

12.1 Example Query

Question:

Which authors are connected because one author's paper cites another author's paper?

In Neo4j, this is naturally written as a graph path:

```
(Author)-[:AUTHORED]->(Paper)-[:CITES]->(Paper)<-[:AUTHORED]-(Author)
```

In PostgreSQL, the same idea requires joining multiple tables:

- paper_authors;
- citations;
- authors.

This is one of the main differences between the two systems.

13 Step 9: Benchmarking

The benchmark script is:

src/benchmark_queries.py

We run:

```
python src/benchmark_queries.py --runs 5 --warmups 1
```

Benchmark method:

- 1 warmup run per query/system;
- 5 measured runs per query/system;
- PostgreSQL measured with `EXPLAIN ANALYZE`;
- Neo4j measured with `PROFILE`;
- raw results saved as CSV.

Results are saved in:

benchmarks/results/benchmark_results.csv
docs/benchmark_results.md

14 Step 10: Benchmark Results

The main result:

- PostgreSQL was faster on 10 out of 11 queries.
- Neo4j was faster on 1 query: author citation network.

Query	PostgreSQL ms	Neo4j ms	Faster
Most cited papers	0.058	4.200	PostgreSQL
Top authors	2.422	8.600	PostgreSQL
Top topics	4.762	15.200	PostgreSQL
Author collaboration	44.358	65.000	PostgreSQL
Citation links	0.183	4.400	PostgreSQL
RAG-related papers	3.515	8.600	PostgreSQL
Two-hop citation paths	1.577	8.400	PostgreSQL
Shared topics	309.447	586.600	PostgreSQL
Shared references	2.171	7.800	PostgreSQL
Paths up to 3 hops	5.104	12.400	PostgreSQL
Author citation network	48.214	37.400	Neo4j

Table 3: Average benchmark results

Interpretation:

- PostgreSQL is very strong for structured tables, sorting, filtering, joins, and aggregations.
- Neo4j is more natural for graph-shaped questions involving paths and relationships.
- The result depends on query type and dataset size.

15 Step 11: Visual Evidence

For the presentation, we collected screenshots showing:

- OpenAlex source and documentation;
- Docker containers running;
- PostgreSQL tables in Adminer;
- SQL query result;
- Neo4j database overview;
- Neo4j Author–Paper–Topic graph;
- Neo4j citation network;
- benchmark chart.

These screenshots are stored in:

`docs/figures/`

16 Step 12: Final Deliverables

The final deliverables are:

- docs/project_proposal.pdf
- docs/project_roadmap.pdf
- docs/project_report.pdf
- docs/OpenAlex_DBMS_Comparison_Sapienza.pptx
- README.md
- database scripts, benchmark scripts, query files, and results.

17 How to Explain the Project in the Presentation

Use this simple storyline:

1. We wanted to compare a relational DBMS and a graph database.
2. We chose OpenAlex because scholarly data contains papers, authors, topics, and citations.
3. We collected AI/RAG-related papers through the OpenAlex API.
4. We cleaned and normalized the data into CSV files.
5. We loaded the same CSV files into PostgreSQL and Neo4j.
6. PostgreSQL represents relationships using tables and joins.
7. Neo4j represents relationships directly as graph edges.
8. We wrote equivalent SQL and Cypher queries.
9. We benchmarked 11 query pairs.
10. PostgreSQL was faster for most structured analytical queries.
11. Neo4j was better and more natural for the author citation network query.
12. The conclusion is that the best database depends on the query shape.

18 What Leo Should Know Before Presenting

Leo does not need to memorize all code. He should understand these core points:

- The same dataset is used in both databases.
- PostgreSQL uses tables and bridge tables.
- Neo4j uses nodes and relationships.
- ETL means collecting, cleaning, and loading data.
- SQL is better for tabular aggregation.

- Cypher is clearer for path traversal.
- The benchmark result is local to our dataset and workload.
- We are not saying one database is always better than the other.

19 How This Connects to the Lecture Slides

This project is useful because it connects directly to topics from the course. We should mention these connections during the presentation because they show that the project is not only a programming exercise.

Lecture topic	Connection to our project
Graph databases	Our OpenAlex data is naturally graph-shaped: authors write papers, papers have topics, and papers cite other papers.
Neo4j property graph	We use <code>Paper</code> , <code>Author</code> , and <code>Topic</code> nodes, with <code>AUTHORED</code> , <code>HAS_TOPIC</code> , and <code>CITES</code> relationships.
Cypher pattern matching	Our Neo4j queries use <code>MATCH</code> patterns, for example <code>author to paper to cited paper to author</code> .
Relational joins	PostgreSQL does not store relationships as edges, so it reconstructs them through joins and bridge tables.
Query processing	Our SQL benchmark queries use selections, projections, joins, grouping, sorting, and aggregation.
Indexes and file organization	PostgreSQL uses primary keys and indexes; Neo4j uses constraints/indexes before graph traversal.
RDF databases	We did not use RDF triples because our approved project is a property graph DBMS comparison, not a semantic web project.
Data warehousing	We did not build a star schema or OLAP project because our topic is DBMS comparison.

Table 4: Course topics connected to the project

Short explanation to say orally:

The course explains that relational databases represent relationships through joins, while graph databases represent relationships directly as edges. Our project applies this idea to OpenAlex data: in PostgreSQL we use bridge tables and joins, while in Neo4j we use graph relationships such as `AUTHORED`, `HAS_TOPIC`, and `CITES`.

20 Possible Professor Questions and Short Answers

Question	Answer
Why OpenAlex?	It is open, has an API, and contains papers, authors, topics, and citations.
Why not use the full dataset?	Full OpenAlex is too large, so we used a focused AI/RAG subset suitable for local testing.
How do you know the comparison is fair?	Both databases are loaded from the same normalized CSV files and the counts match.
Why is PostgreSQL faster on most queries?	Many queries are aggregations and joins on a small structured dataset, which PostgreSQL handles well.
Why use Neo4j then?	Neo4j is clearer for graph-shaped questions, paths, and network exploration.
What is the main conclusion?	PostgreSQL is strong for structured analysis; Neo4j is strong for relationship-heavy traversal.

Table 5: Possible questions and short answers

21 Commands to Reproduce the Project

From the repository root:

```
python src/collect_openalex.py --per-term 75 --per-page 75
```

```
docker compose up -d
```

```
docker exec openalex-postgres psql -U openalex -d openalex_ai -f /sql/schema.sql
```

```
docker exec openalex-postgres psql -U openalex -d openalex_ai -f /sql/load.sql
```

```
docker exec openalex-neo4j cypher-shell -u neo4j -p openalex123 -f /cypher/constraints.cypher
```

```
docker exec openalex-neo4j cypher-shell -u neo4j -p openalex123 -f /cypher/import.cypher
```

```
python src/benchmark_queries.py --runs 5 --warmups 1
```

```
python src/generate_benchmark_charts.py
```

22 Final Simple Conclusion

The project is complete because it includes the full pipeline:

```
OpenAlex API -> raw JSONL -> cleaned CSV -> PostgreSQL + Neo4j  
-> equivalent queries -> benchmarks -> charts -> report -> presentation
```

The final message is:

PostgreSQL is better for structured tabular analysis in our current benchmark, while Neo4j is more natural for graph relationships such as citations and author networks.